INTERVIEW PREPARATION GUIDE

AI-Optimized Fmax Search for Semiconductor Test

Everything you need to explain this project clearly for 45–60 minutes and answer any question about it.

The project in one line: a machine-learning model predicts each chip's maximum speed from measurements it already has, so the tester only checks a small window instead of searching blindly. That cuts frequency tests by **40%**, with **zero** chips sold at a speed they can't handle.

All results come from the supplied synthetic dataset (6,000 chips). Say this in the interview, every time you quote a number.

Contents

1. How to use this guide
2. Your 60-second pitch
3. The problem, in plain words
4. The idea: predict, then measure
5. The data
6. The model
7. Uncertainty: the 95% window
8. The safety net: out-of-distribution fallback
9. The search algorithm
10. Results: the numbers to know
11. Binning and the guard band (your "weak spot")
12. What you built: system and tools
13. The live demo script
14. Your 45–60 minute talk plan
15. Likely questions and good answers
16. Tough questions
17. Limitations and next steps
18. Glossary
19. One-page cheat sheet

1 How to use this guide

- **Day 1:** read sections 2–11 once, slowly. Don't memorize yet; just understand the story.
- **Day 2:** learn the pitch (section 2) and the cheat sheet (section 19) by heart. Practise the demo (section 13).
- **Day 3:** say the full talk out loud with a timer (section 14). Then have your friend ask you the questions in sections 15–16.
- **Interview morning:** re-read the cheat sheet only.

TIP

Interviewers don't expect you to know everything. They want to see that you understand **why** each decision was made, that you're honest about limits, and that you think about real production use. This guide is organized around those three things.

2 Your 60-second pitch

Learn this almost word for word. Use it to open the interview and whenever someone asks "So, what did you build?"

SAY IT LIKE THIS

"Chip makers have to find each chip's maximum working frequency, called Fmax, to sort chips into speed grades. Today the tester does that with a blind search: it tries one frequency after another, about six or seven tries per chip, at every voltage and temperature setting. That takes a lot of expensive tester time.

I built a system that uses measurements the chip already has, like on-chip ring oscillators and leakage current, to predict its Fmax. A calibrated 95% confidence window then tells the tester exactly where to look. The tester still measures the real Fmax, just inside that small window.

On a 6,000-chip dataset, that cut the number of tests by 40%, from 6.45 to 3.87 per chip, with zero chips placed in a faster grade than they can handle. If a chip looks unusual, the system falls back to the normal full search, so it's safe by design. I delivered it as a Python pipeline, a REST API and an interactive dashboard."

Three things to say early

1. **The model never decides the grade.** It only decides where the tester looks. The grade always comes from a real measurement.

2. **It's safe when wrong.** A bad prediction costs a few extra tests, never a wrong answer.
3. **The data is synthetic.** The results show the method works; they are not production numbers.

3 The problem, in plain words

What is Fmax?

Every chip comes out of the factory slightly different, because of tiny variations in manufacturing. Some chips can run fast, some only slower. **Fmax** is the highest clock frequency at which a chip still works correctly. In this dataset, Fmax averages about 1,988 MHz and ranges from about 1,616 to 2,330 MHz.

What is speed binning?

Chips are sorted into grades, called **bins**, by their Fmax. Faster chips sell for more. Here the bins are 1,800 / 1,900 / 2,000 / 2,100 / 2,200 MHz. A chip that can't reach 1,800 is rejected.

How does the tester find Fmax today?

The Automatic Test Equipment (ATE) sets a clock frequency, runs test patterns, and checks pass or fail. Then it picks another frequency and repeats. The two usual methods:

- **Linear sweep:** start low, step up 10 MHz at a time until the chip fails. About **40** tests per chip.
- **Binary search:** test the middle, then the middle of the half that's left, and so on. About **6–7** tests per chip. This is the industry default.

Why does this matter?

- Every test step = changing the clock (about 5 ms) + running the patterns (about 25 ms).
- The whole search is repeated at every voltage/temperature corner.

- For millions of chips, tester time is a major cost. Testers are expensive machines.

Easy analogy to use: it's the "guess my number" game. If the number is between 1 and 81, you need about 7 guesses with smart halving. But if a friend whispers "it's between 40 and 50", you need only about 3. My model is that friend.

4 The idea: predict, then measure

Before the frequency test, every chip already has cheap measurements. The model turns them into a prediction plus a window, and the tester searches only inside the window.

1 Measure	Read 15 values the chip already has: 10 ring oscillators, leakage current (IDDQ), minimum voltage (Vmin), and die position on the wafer.
▼	
2 Predict	A linear regression model estimates Fmax. It takes about 1.3 microseconds per chip.
▼	
3 Bound	Conformal prediction adds a calibrated 95% window: prediction ± 48 MHz.
▼	
4 Search	The tester runs binary search only inside the window. If the real Fmax turns out to be outside it, the search extends past the edge.
▼	
5 Bin	The grade comes from the **measured** Fmax, with an 8 MHz safety margin (guard band).

Fallback: if a chip's measurements look unusual compared with the training data, the prediction is not trusted. That chip gets the normal full binary search.

SAY IT LIKE THIS

"The key design decision is that the model is a **guide**, not a **judge**. It tells the tester where to look, but the final answer is always a real measurement. That's why it's safe even when the model is wrong."

This is called "prediction-windowed search" in the technical document the project followed. It recommends it as the safe default for production.

5 The data

6,000 chips, 25 columns. It's a synthetic (simulated) dataset that came with the project. It was clean: no missing values, no duplicates. Two extreme values were found (one in `ro_03`, one in `ro_09`); they were flagged but kept, because removing data you don't like is bad practice.

The fixed splits

Split	Chips	Used for
Train	1,500	Fitting and choosing the model
Calibration	500	Only for sizing the 95% window
Test	4,000	Final results. Never used for any decision.

The 15 inputs the model may use

Feature	What it is	Why it predicts speed
ro_01 – ro_10	Ring oscillators: tiny on-chip circuits whose frequency tracks transistor speed	Strongest signal: fast transistors = fast oscillators = fast chip
iddq_norm	Leakage current when idle	Fast transistors leak more
vmin_norm	Lowest voltage the chip works at	Slow chips need more voltage
die_x, die_y, radius_sq	Position on the wafer	Chips near the wafer edge often behave differently

Data leakage: the biggest risk

The file also contains 10 columns the model must **never** see: the true Fmax (the answer), an existing prediction, existing window bounds, the true bin, past search results, the chip ID and the split label. If any of these got into training, results would look amazing and be fake.

SAY IT LIKE THIS

"I used a strict allowlist: only the 15 measurement columns can reach the model, and the code raises an error if anything else slips in. I also wrote tests that check every forbidden column individually."

6 The model

I compared four models, as the project brief asked. One small extra feature step: the mean, standard deviation, min and max of the 10 ring oscillators. Averaging 10 noisy readings gives a cleaner estimate of the chip's true speed.

Model	CV RMSE	Test RMSE	Speed
Linear regression ✓	**24.4**	**25.4**	**1.3 µs**
Random forest	25.3	26.8	55.5 µs
Gradient boosting	25.4	26.5	11.0 µs
XGBoost	25.5	26.6	9.7 µs
Prediction already in the file	–	26.9	–

RMSE in MHz (lower is better). Speed = prediction time per chip.

How I chose

- 5-fold cross-validation (CV) on the training split only. The test split was never used to choose. That keeps the final numbers honest.
- Rule: the lowest CV error; if several are within 0.25 MHz, pick the fastest.
- Linear regression won on accuracy outright, and it's also the fastest and simplest.

Final model quality (4,000 test chips)

0.941
R^2 (target ≥ 0.90)

25.4 MHz
RMSE, about 1.3% of Fmax

20.3 MHz
Mean absolute error

1.02%
Mean absolute % error

SAY IT LIKE THIS

"Interestingly, the simple linear model beat the tree models. That tells me the relationship between the sensors and Fmax is mostly linear in this data, and with only 1,500 training chips the tree models can't gain anything. A linear model is also a big plus in production: it's just a few coefficients, so it can be embedded directly in the test program."

7 Uncertainty: the 95% window

A prediction alone isn't enough to steer a tester. You need to know how wrong it might be. I used **split conformal prediction**.

How it works, in plain words

1. Take the 500 calibration chips. The model never trained on them.
2. For each one, measure how far off the prediction was.
3. Sort those errors and pick the size that covers 95% of them. Here: **48.3 MHz**.
4. For any new chip, the window is prediction ± 48.3 MHz (about 97 MHz wide, roughly 10 tester steps).

(Technically it's the $\lceil (n+1)(1-\alpha) \rceil$-th smallest error, the 476th of 500, a small correction that makes the guarantee exact for a finite sample.)

Result

On the 4,000 unseen test chips, the true Fmax fell inside the window **94.7%** of the time. The target was 95%. ✓

Why conformal and not just "± 2 standard deviations"?

- It makes no assumption that errors follow a bell curve.
- It comes with a mathematical guarantee of at least 95% coverage, as long as new chips look like the calibration chips.
- It works with any model, so I could swap models without redoing it.

I also tried a smarter version

"Normalized" conformal gives each chip its own window width, based on how hard that chip is to predict. It reached 95.2% coverage, but the windows were wider on average (104 vs 97 MHz), because errors in this data are similar for all chips. So I kept the simple version.

TIP

"I tried something more advanced, measured it, and kept the simpler option because the data showed it was better" is a great thing to say in an interview. It shows judgement.

8 The safety net: out-of-distribution fallback

The 95% guarantee only holds if new chips look like the training chips. If a chip comes from a strange batch, or a sensor is broken, the prediction can't be trusted. So before using the model, every chip gets two checks:

1. **Range check:** is any of the 15 values far outside what we saw in training (more than 10% beyond the min or max)?
2. **Mahalanobis distance:** is the *combination* of values unusual? For example, ring oscillators saying "fast chip" while leakage says "slow chip". Each value can look normal on its own while the combination is strange. The threshold is the 99.9th percentile of training chips.

A chip that fails either check (or has missing values) skips the model and gets the full binary search.

Result	Value
Test chips sent to full search	0.6% (24 chips)
Artificially shifted chips caught	78.8%
Cost of the fallback on average tests	+0.03 tests/chip

SAY IT LIKE THIS

"Even when the detector misses an unusual chip, nothing breaks. If the real Fmax is outside the window, the search notices at the window edge and keeps going. So a missed detection costs a few extra tests, never a wrong measurement. The detector is a second layer of safety, not the only one."

9 The search algorithm

The simulated tester

There's no real tester, so I built a simulator that follows the technical document's settings:

- Frequencies from 1,600 to 2,400 MHz in 10 MHz steps (81 settings).
- A chip "passes" if the frequency is below its true Fmax, plus random test noise of about 4 MHz. Real testers are noisy near the boundary.
- Each test costs 30 ms (25 ms patterns + 5 ms clock switch).
- Every strategy runs on the same 4,000 test chips with the same noise, so the comparison is fair and reproducible.

The windowed search, step by step

1. Snap the 95% window to the 10 MHz grid.
2. Binary search inside the window.
3. If the top of the window **passes**, the real Fmax is higher: jump upward in doubling steps (1, 2, 4…) until a fail, then binary search the gap.
4. If everything in the window **fails**, the real Fmax is lower: binary search below the window.

That's why it's always correct: the window only decides where to start looking.

A real example from the dashboard (chip U02001)

	Binary search	AI-assisted
Starts at	Middle of 1,600–2,400	Inside window 1,930–2,040
Tests	2000✗ 1790✓ 1890✓ 1940✓ 1970✗ 1950✗	1980✗ 1950✓ 1960✗
Count	**6 tests**	**3 tests**
Measured	1,940 MHz	1,950 MHz
Grade	1,900 bin	1,900 bin

True Fmax was 1,944.8 MHz. The two readings differ by one step because of the 4 MHz tester noise, which is exactly why the 8 MHz guard band exists. Both give the same, correct grade.

10 Results: the numbers to know

Search cost (4,000 test chips, one corner)

Strategy	Avg tests	95th pct	Time/chip
Linear sweep	40.5	58	1,214 ms
Binary search (today)	6.45	7	194 ms
Window shipped with data	3.94	7	118 ms
ML window only	3.84	4	115 ms
ML window + fallback (final)	**3.87**	**4**	**116 ms**

40%
fewer tests than binary search (target ≥ 30%)

0
chips over-graded (sold faster than they can run)

4%
of chips needed the window extended

5.7 MHz
measurement error, same as binary search

The PRD acceptance criteria

Criterion	Target	Result
R^2	≥ 0.90	0.941 ✅
Interval coverage	≈ 95%	94.7% ✅
Test reduction	≥ 30%	40.0% ✅
Bin accuracy	≥ 95%	90.5% ⚠️ (section 11)
Data leakage	None	Enforced in code ✅
Fallback tested	Yes	✅

What it could be worth (illustrative only)

Using the technical document's example: 3 voltage corners at system-level test (0.4 s per step), 20 million chips a year, $0.004 per tester-second. Then 2.6 fewer steps per corner ≈ **3.1 s saved per chip**, about **17,200 tester-hours** or **$248k a year**. Always say these are assumed numbers.

11 Binning and the guard band (your "weak spot")

One target was missed: bin accuracy is **90.5%**, against a target of 95%. Expect this question. Prepare it well and it becomes a strength.

Why it happens

- The tester is noisy (±4 MHz). A chip at 1,905 MHz might "pass" 1,910 once by luck.
- So a chip only gets a grade if it passes **grade + 8 MHz** (the guard band). That protects customers.
- Chips sitting just above a grade edge get placed one grade lower. That's the ~9.5% "under-binned".
- **Plain binary search, with no AI at all, also scores 90.5%.** So the model is not the cause.

Guard band	Bin accuracy	Over-graded
0 MHz	97.3%	1.43% ✗
8 MHz (used)	**90.5%**	**0 ✓**
12 MHz	80.5%	0

SAY IT LIKE THIS

"I could hit 97% by removing the guard band, but then 1.4% of chips would be sold as faster than they are. Those are customer failures. In semiconductors, an over-graded chip is far worse than an under-graded one, so I kept the safe setting and reported the trade-off honestly. Where to set the guard band is a business decision for the product team, not a modelling decision."

Why not grade chips from the prediction alone?

Skipping the test entirely would be even faster, so I checked it:

Safety margin	Over-graded	Under-graded
None	9.6%	10.8%
1σ (25 MHz)	2.2%	28.4%
3σ (75 MHz)	0%	71.9%

Almost 1 in 10 chips would be over-graded. Making that zero would push 72% of chips down a grade (lost revenue). The model's 25 MHz error is too big compared with the 100 MHz grades, so a real measurement is needed. That proves the design choice.

12 What you built: system and tools

Tech stack

Python · pandas, NumPy · scikit-learn, XGBoost · Streamlit (dashboard) · FastAPI (API) · Plotly, Matplotlib · pytest · joblib · Git/GitHub

Code structure (the `fmax` package)

File	Job
`config.py`	All settings: features, forbidden columns, tester grid, bins
`data.py`	Load and validate the data; fixed splits
`features.py`	Leakage guard; ring-oscillator summary features
`models.py`	Train, compare, select and save models
`uncertainty.py`	Conformal windows; OOD detector
`search.py`	Tester simulator; all search methods; binning
`pipeline.py`	Runs everything end to end; writes results
`service.py`	One prediction service shared by API and dashboard

Interfaces

- **Pipeline:** `python -m fmax.pipeline` rebuilds everything in about 53 seconds, with identical results every run (fixed random seeds).
- **Dashboard:** 9 pages (overview, models, uncertainty, search, binning, chip explorer, live prediction, data, reports).
- **REST API:** `/predict` (window + decision), `/simulate` (full test run), `/metrics`, `/model`, `/health`.
- **Tests:** 30 automated tests covering data checks, leakage, search correctness, conformal coverage and the API.
- **Versioned model:** model + window + OOD detector saved together, with a data fingerprint and creation time.

SAY IT LIKE THIS

"I separated the shared logic into one service, so the dashboard and the API can never disagree about a chip. In a real factory, the tester would call the API, or use the model coefficients directly, before the frequency search."

13 The live demo script (about 5 minutes)

Open the dashboard before the interview starts. Have the deployed link or `streamlit run dashboard/app.py` ready, with a backup screenshot in case of network trouble.

1. **Overview page (1 min):** point at the headline numbers: R^2 0.941, 40% fewer tests, 0 over-bin escapes. Show the acceptance table, including the one target not met.
2. **Search optimization (1 min):** the bar chart. Binary 6.45 vs AI-assisted 3.87. Show the pie: most chips resolve inside the window.
3. **Device explorer (2 min):** the strongest moment. Pick chip U02001. Show the two charts side by side: 6 tests vs 3. Then switch the filter to "Window extended" to show the search recovering when the prediction is off. Then "OOD fallback" to show the full search on an unusual chip.

4. **Predict new devices (1 min):** change a value in the form, for example set `ro_01` to 25. Watch it flip to "out of distribution → full search". That proves the safety net live.

> **TIP**
> Let the interviewer click if they want. Say what you're doing before you click, and keep the demo short. The talk carries the story; the demo is the proof.

14 Your 45–60 minute talk plan

The slide deck (`docs/AI_Fmax_Search_Final_Presentation.pptx`, 13 slides, with speaker notes) follows this order.

Min	Part	Key message
0–3	Pitch	Section 2, word for word
3–8	Problem	Fmax, bins, blind search, cost. Use the number-game analogy.
8–12	The idea	Predict → window → measure. Model is a guide, not a judge.
12–17	Data	Splits, 15 features, leakage guard
17–23	Model	4 models, CV selection, why linear won
23–28	Uncertainty + OOD	94.7% coverage; two-layer safety
28–33	Search + results	6.45 → 3.87, 40%, U02001 example
33–38	Binning	The 90.5% story; why measure, not predict
38–43	Demo	Section 13
43–47	System	Pipeline, API, dashboard, 30 tests
47–50	Limits + next steps	Section 17
50–60	Questions	Sections 15–16

TIP

If time is short, cut the System part first, then shorten Data. Never cut the Binning story or the limitations: they show maturity.

TIP

Pause after each part and ask "Any questions so far?" It turns a lecture into a conversation, and interviewers love that.

15 Likely questions and good answers

Q: Why not just use binary search? It's only 6–7 tests.
At one corner, yes. But it's repeated at every voltage/temperature corner, for millions of chips, and at system-level test each step can take close to half a second. Saving 2.6 steps per corner adds up to thousands of tester-hours.

Q: Why did linear regression beat XGBoost?
The sensor-to-Fmax relationship is mostly linear in this data, and with 1,500 training chips the trees can't find extra structure, while they approximate straight lines with steps. I chose by cross-validation, not by preference. Linear is also faster and easy to embed in a test program.

Q: Why cross-validation if you already have a test set?
If I used the test set to pick the model, the test results would be optimistic. CV on the training data picks the model; the test set is only for the final, honest report.

Q: What does the calibration set do that the test set can't?
It sizes the window. It has to be data the model never trained on, otherwise the errors look smaller than they really are. And it can't be the test set, or the coverage number would be self-fulfilling.

Q: What is R^2 in simple terms?
The share of the chip-to-chip variation in Fmax that the model explains. 0.941 means it explains 94% of it; the rest is local variation the sensors can't see.

Q: What happens when the prediction is wrong?
The search hits the window edge, sees a pass at the top (or a fail at the bottom), and extends. You lose a few tests, never accuracy. It happened for 4% of chips.

Q: How did you prevent data leakage?
An allowlist of 15 measurement columns, enforced in code. Anything else raises an error, and tests check each forbidden column one by one.

Q: What's Mahalanobis distance?
A distance that takes into account how features normally move together. It flags chips whose combination of values is unusual, even if each value alone looks fine.

Q: How does tester noise affect binary search?
One wrong pass/fail early on sends binary search into the wrong half for good. Both methods face the same noise in my simulation. The 8 MHz guard band protects the grade decision against it.

Q: How would this run on a real tester?
Three options: embed the linear coefficients in the test program; call the model on the tester host via the API; or compute predictions upstream after wafer sort and send them with the lot. Inference takes about a microsecond, so it adds no meaningful time.

Q: How would you monitor it in production?
Keep a few percent of chips on the full search as an audit sample, giving unbiased labels. Then track errors on them over time, interval coverage, fallback rate and window-extension rate. A rise in any of these triggers retraining.

Q: Why a Streamlit dashboard and a FastAPI backend?
The dashboard is for engineers to explore and trust the results. The API is how a machine, such as the tester cell, would ask for a window. Both use the same service code, so they always agree.

Q: How do you know the results are reproducible?
Every random step is seeded, including a per-chip tester noise stream. Re-running the pipeline gave identical results in 53 seconds, and the dashboard replays the exact benchmark runs.

Q: What was the hardest part?
Making the system safe, not just accurate. Accuracy alone was easy to reach. The real work was the edge-extension logic, the fallback and showing the binning trade-off, so a wrong prediction can never become a wrong grade.

Q: What would you improve with more time?
Real data, multi-corner window reuse (use the result at one voltage to narrow the next), and a Bayesian search that handles noisy tests better.

16 Tough questions

Answer calmly and honestly. Admitting a limit and explaining how you'd handle it scores better than defending.

Q: The data is synthetic. Doesn't that make the results meaningless?
"They show the method works and how the pieces fit together, not what a real product will save. The pipeline is built to be re-run on real data: the same validation, model comparison, calibration and benchmark. On real data I'd expect the numbers to change, which is exactly why the system measures its own coverage and has a fallback."

Q: You missed the 95% bin-accuracy target.
Use the section 11 answer: binary search scores the same; the guard band causes it; removing it creates 1.4% over-grading; it's a business trade-off, and I chose safety.

Q: Zero escapes in 4,000 chips. Can you claim it's safe?
"Not at a parts-per-million level. Zero out of 4,000 only says the escape rate is below about 750 ppm, with 95% confidence (rule of three: 3 ÷ 4,000). Proving ppm-level safety needs far more chips, through shadow mode and audit sampling in production."

Q: Your OOD detector only caught 79% of shifted chips.
"Right, and that's why it isn't the only safeguard. The 21% it misses still get a correct measurement through the window-edge check; they just cost a few extra tests. I'd improve it with lot-level checks and drift monitoring."

Q: The dataset already had a window that saved 39%. What did you add?
"Fair point: my ML window is only slightly better (3.84 vs 3.94 tests) and needs fewer window extensions (4% vs 10.5%). What I added is the system around it: a calibrated window with a known coverage guarantee, out-of-distribution fallback, leakage-safe training, the full comparison, the binning analysis, and a deployable API and dashboard."

Q: Did you build all of this yourself? Did you use AI tools?
Be honest about how you worked and which tools you used. Most companies accept AI-assisted coding today. What matters is that you understand and can defend every decision in this guide. If you can explain section by section why each choice was made, you own the work.

WATCH OUT

Never invent numbers or experiments you didn't run. If you don't know, say "I didn't test that, but here's how I would." That answer is respected.

17 Limitations and next steps

Limitations (say them before they ask)

- Synthetic data: the relationships and noise levels are assumptions.
- Only one voltage/temperature corner was simulated.
- 4,000 chips can't prove ppm-level escape rates.
- The 95% bin-accuracy target needs a business decision on the guard band.
- The OOD check looks at single chips, not whole lots drifting slowly.

Path to production (from the technical document)

1. **Shadow mode:** run the model on real lots, but keep using the normal search. Compare.
2. **Limited release:** use the window on a few lots, side by side with the reference.
3. **Audit sampling:** about 60 fully searched chips per monitoring period, enough to detect a 10 MHz error drift.
4. **Drift monitoring:** watch errors, coverage, fallback and window-extension rates; retrain on alarms.
5. **Multi-corner reuse:** use one corner's result to narrow the next for extra savings.
6. **Rollback:** one switch returns to the normal flow at any time.

SAY IT LIKE THIS

"I'd never switch this on directly. I'd run it in shadow mode first, prove it on real lots, and keep a small audited sample on the full search forever, so we always know if it starts drifting."

18 Glossary

Term	Meaning
Fmax	Highest clock frequency a chip runs correctly at
ATE	Automatic Test Equipment: the machine that tests chips
Speed bin	Speed grade a chip is sold as
Guard band	Safety margin: must pass bin + 8 MHz to get that bin
Over-bin escape	A chip graded faster than it can really run. The worst outcome.
Under-bin	A chip graded slower than it could be. Lost revenue, but safe.
Corner	One voltage + temperature combination tested
Ring oscillator (RO)	Tiny on-chip circuit whose speed tracks transistor speed
IDDQ	Leakage current when the chip is idle
Vmin	Lowest supply voltage the chip works at
Data leakage	Answer information sneaking into model inputs
Cross-validation	Train on part of the data, check on the rest, rotate 5 times
RMSE / MAE	Typical prediction error in MHz (RMSE punishes big errors more)
R^2	Share of variation explained (1.0 = perfect)
Conformal prediction	Window sized from real errors on held-out data, with a coverage guarantee
Coverage	How often the true value falls inside the window
OOD	Out-of-distribution: a chip unlike the training data
Mahalanobis distance	How unusual a combination of values is
Shadow mode	Model runs and logs, but doesn't make decisions yet
Audit sampling	A few chips always get the full search, to check the model
PRD	Product Requirements Document: the project brief

19 One-page cheat sheet

Fact	Number
Chips (train / calib / test)	1,500 / 500 / 4,000
Model inputs	15
Chosen model	Linear regression
R^2 / RMSE / MAE	0.941 / 25.4 / 20.3 MHz
Prediction time	1.3 µs/chip
95% window	± 48 MHz
Coverage achieved	94.7%
Tests: linear / binary / AI	40.5 / 6.45 / 3.87
Saving vs binary	40%
Time per chip: binary → AI	194 → 116 ms
Window extended	4% of chips
OOD fallback rate	0.6%
Shifted chips caught	79%
Over-bin escapes	0
Bin accuracy (binary also)	90.5%
No guard band	97.3%, but 1.4% over-graded
Prediction-only grading	9.6% over-graded
Grid / noise / step time	1,600–2,400 by 10 MHz / 4 MHz / 30 ms
Bins / guard band	1,800–2,200 by 100 / 8 MHz
Automated tests / rebuild time	30 / 53 s

If you remember only three sentences:

1. The model tells the tester **where to look**; the grade always comes from a **measurement**.
2. **40% fewer tests** than binary search, with **zero** over-graded chips.
3. Wrong predictions cost **extra tests, never wrong answers**; unusual chips get the **full search**.